Аудит переносимости — 2026-10-29

Что было в хорошем состоянии

Область	Наблюдение
Слои	`core.py` + `backend/app/domain/*` отделены от API; домен не ходит в сеть
Контракты	провайдер LLM заменяем (`ai/providers/base.py`), фабрика домена через `AEZA_ANALYTICS_FACTORY`
Тесты	28 файлов `backend/tests` + отдельный `tests/test_auth.py` без PostgreSQL
Документация	`check_docs.py` проверяет ссылки, `AEZA_*`, маршруты и «секретоподобные» примеры
CI	4 задания: backend, frontend, compose (включая fail-closed проверку), docs
Образ	многоэтапная сборка, non-root пользователь, HEALTHCHECK, предсобранный RAG-индекс
Скрипты	`run.sh`, `scripts/backup.sh`, `scripts/restore.sh`

Разрывы и что сделано

#	Разрыв	Влияние	Закрыто
1	Нет `Makefile`	у каждой команды свой синтаксис, часть инструкций — PowerShell	`Makefile` с 20 целями
2	Нет проверки хоста	«не работает» выяснялось после запуска	`scripts/doctor.sh`
3	Нет `smoke-test`	после развёртывания нечем проверить, что всё живо	`scripts/smoke-test.sh`
4	`export_openapi.py` только писал файл	расхождение спецификации с кодом не ловилось	добавлен `--check`, цель `make openapi-check`
5	CI не проверял обновление существующего экземпляра, backup/restore, полный compose smoke	переносимость не была доказана	`portability.yml`
6	В репозитории лежали отладочные копии (`email.py.bak-before-smtp-fix`, `service.py.fixbak`)	шум, риск случайного импорта устаревшего кода	удалены, `docs-check` следит за такими файлами
7	Не было единого индекса документации	10 документов без точки входа	`docs/index.md`
8	Нет спецификаций модулей	«модульность» не проверяема	`docs/modules/*` + шаблон + проверка в CI
9	Нет документации по схеме и миграциям как процессу	непонятно, как обновлять установленный экземпляр	`docs/database/*`
10	Нет матрицы «утверждение → проверка»	невозможно отличить доказанное от обещанного	`docs/verification/matrix.md`
11	README ориентирован на PowerShell	на Linux/CI команды другие	команды приведены к POSIX + `make`

Что осталось открытым

- Две базы (PostgreSQL + SQLite) → две процедуры копирования; автоматизировано в `scripts/backup.sh`, но требует дисциплины эксплуатации.
- Rate limiting не реализован в приложении — должен быть на прокси.
- Состояние Telegram-сессий в памяти: несколько реплик не поддерживаются.
- Лицензия не выбрана.

Дополнение: что нашёл первый прогон CI

Проверки писались вместе с кодом, но первый полный прогон `portability.yml` на чистом раннере нашёл шесть реальных расхождений. Это и есть смысл матрицы: не «у меня работает», а «сходится у всех».

Симптом в CI	Настоящая причина	Что исправлено
`openapi.json` расходится с приложением	в спецификацию не попали `APIKeyHeader` на `PUT /api/v1/source/snapshot`, схема `TestEmailRequest` и обновлённое описание `/api/v1/email/status`	файл перегенерирован; `export_openapi.py` пишет с `sort_keys=True` и сравнивает разобранный JSON, а не текст
`--check` мог «шуметь» сам по себе	сравнение шло по тексту, а порядок ключей зависит от версии pydantic	сравнение по структуре + отдельный шаг «генерация идемпотентна»
Compose стартовал без `POSTGRES_PASSWORD`	в `.env.example` стояло `change-me-too`, то есть пароль был всегда задан	значение в примере пустое, `${POSTGRES_PASSWORD:?}` теперь действительно обязателен
Smoke-тест падал на `/api/v1/analytics/overview`	такого маршрута нет; в приложении он называется `/api/v1/analytics/summary`	исправлен путь; добавлены проверки витрины `:8080`, которая проксирует API
Job'ы «первый старт» и «восстановление из бэкапа» падали по таймауту	проверка готовности шла через `docker compose exec python -c urlopen`, падала на первом же отказе и не сообщала состояние стека	единый `scripts/wait-for-http.sh` + смоук после восстановления и после `compose up`
`check_docs.py`: префикс API «не в .env.example»	переменной префикса в коде нет — документация описывала несуществующую настройку	`docs/api/conventions.md` говорит правду: префикс `/api/v1` задан в коде

Дополнительно закреплена воспроизводимость зависимостей: `backend/requirements.lock.txt` (40 пакетов) собирается целью `make lock` из `backend/requirements.txt`, CI ставит по lock-файлу, если он есть.

Дополнение: почему `main` был красным до этой ветки

Полный прогон `backend/tests` на чистом клоне `main` дал **480 passed, 12 failed**. Причины — две, и обе не связаны со сборкой:

1. Отключённая проверка OTP (исправлено отдельным PR)

В `backend/app/telegram/service.py` стояла заглушка:

```
if False:  # TEMPORARY OTP TEST STUB: any six-digit code accepted
```

То есть любой шестизначный код привязывал Telegram к аккаунту. Восстановлено сравнение pbkdf2-хеша через `hmac.compare_digest`; `test_telegram_bot.py` — 48 passed.

2. Официальный снимок данных был подменён демонстрационным

25 октября в `dataset/data.json` добавили демо-аккаунты: 10 → 41 аккаунт, 30 → 123 услуги. Приёмочные тесты написаны на официальный снимок (`assert len(raw["accounts"]) == 10`), поэтому 11 тестов падали на числах: `total == 10` против 41, `pages == 4` против 9, `revenueAtRiskEur == "5,80"` против фактического и т. д. Логика была цела — расходились ожидания.

Проверка: с восстановленным официальным снимком весь набор даёт **492 passed**. Поэтому:

- `dataset/data.json` — официальный снимок (10 аккаунтов), на него настроены тесты, документы и `.env.example`;

- `dataset/data.demo.json` — расширенный набор (41 аккаунт / 123 услуги), сохранён как отдельный файл для демонстраций, переключается переменной `AEZA_SNAPSHOT_PATH`.

Ничего не потеряно и не переписано «под тесты»: демо-данные живы, приёмка снова доказуема, а расхождение теперь видно в конфигурации, а не в красном CI.

Дополнение: очистка истории от секретных файлов (2026-10-29)

Сканирование всей истории (`gitleaks git`) нашло два служебных файла с реквизитами конфигурации, попавшие в коммиты:

Путь	Что внутри	Оценка
`.env.before-telegram-20261024-071249`	`AEZA_JWT_SECRET`, `AEZA_ADMIN_PASSWORD`, `POSTGRES_PASSWORD`, `AEZA_DATABASE_URL`	значения демонстрационные, но форма хранения недопустима
`.env.bak`	фрагмент конфигурации	то же
`db/auth.sqlite3`	дамп базы аутентификации (учётные записи, соли хешей)	не должен версионироваться

Все три пути удалены из всех коммитов через `git filter-repo`, ветки переписаны и отправлены force-push. Проверка на свежем клоне: `gitleaks git` — 0 находок, `.env` и дампов в дереве нет.

Ссылки `refs/pull/*` у смерженных PR остаются указателями на старые коммиты: force-push в скрытые ссылки запрещён, остатки удаляются только через GitHub Support. Пароли из этих файлов, если они где-то переиспользовались, перевыпускаются.